Reviewer Pack — Minimal Rank of Geometric Flows vs Resolution Proof Length f...

Entry 22ad113ce145 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 20:54:47 UTC

Minimal Rank of Geometric Flows vs Resolution Proof Length for Tseitin Formulas

Entry ID: `22ad113ce145` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-24 20:54:47 UTC

1. Conjecture

Field A (mathematical branch): Geometric Flow Theory **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity (for Tseitin Formulas)

Statement:

> ['For any given Tseitin formula G on n variables, the resolution proof length of G is at least 2^(Ω(rank(F(G)))), where F(G) is a flow function associated with G and rank(F(G)) is its minimal rank as determined by the Geodesic Flow Invariant.', 'The minimal rank of the flow function F(G) can be computed in polynomial time.']

Rationale (proposer's reasoning):

> ['Geometric flows have been used to study geometric properties of systems, and their invariants have been known to capture essential characteristics of the system. If a deep connection between the resolution proof length and the geometric flow invariant exists, it could provide new insights into the complexity of Tseitin formulas.', 'This conjecture suggests that the rank of the associated flow function could serve as an indicator for the difficulty of proving satisfiability, which has not been explored in the context of resolution proofs.']

Taxonomy category: `TSEITIN_RES` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `fb31d8d7af995f17`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all Tseitin formulas G with n variables (n ≤ 40), the ratio of the resolution proof length to 2^(minimal rank of F(G)) across 30 seeds is greater than or equal to a constant c. The conjecture is falsified if this ratio is less than c for any seed.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	SAFE	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.70	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - geometric flow AND resolution proof complexity Tseitin formulas - minimal rank flow function AND Tseitin formula resolution proofs - Geodesic Flow Invariant AND resolution proof length lower bound

Top relevant hits considered: - [http://arxiv.org/abs/1908.03540v2] Enriching MRI mean flow data of inclined jets in crossflow with Large Eddy Simulations - [http://arxiv.org/abs/2301.01582v1] Study on the Resolution of Large-Eddy Simulations for Supersonic Jet Flows - [http://arxiv.org/abs/2009.01541v1 Sediment transport under oscillatory flows - [http://arxiv.org/abs/1405.4784v1] An Explicit Formula For The Divisor Function - [http://arxiv.org/abs/2110.02266v1] Time-averaged velocity and scalar fields of the flow surrounding a group of cylinders - [http://arxiv.org/abs/1902.07351v2] Phase-field simulation of core-annular pipe flow - [http://arxiv.org/abs/1810.09998v2] Contributions to the study of Anosov Geodesic Flows in Non-Compact Manifolds - [http://arxiv.org/abs/1307.2729v3] On the question of diameter bounds in Ricci flow

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_tseitin_formula(n):
        variables = [f'x{i}' for i in range(1, n+1)]
        clauses = []
        for i in range(1, n+1):
            clauses.append([variables[i-1]])
            clauses.append([-variables[i-1], f'y{i}'])
            clauses.append([f'y{i}', -f'y{i+1}'])
        return variables, clauses

    def dpll(clauses, assignment):
        if not clauses:
            return True
        unit_clause = next((c for c in clauses if len(c) == 1), None)
        if unit_clause:
```

```
        literal = unit_clause[0]
        new_assignment = assignment.copy()
        new_assignment[literal] = True
        if dpll([c for c in clauses if literal not in c and -literal not in c], new_assignment
            return True
        new_assignment[literal] = False
        if dpll([c for c in clauses if literal not in c and -literal not in c], new_assignment
            return True
        return False
    pure_literal = next((l for l in assignment if all(l not in c or -l in c for c in clauses))
    if pure_literal is not None:
        new_assignment = assignment.copy()
        new_assignment[pure_literal] = True
        if dpll([c for c in clauses if pure_literal not in c and -pure_literal not in c], new_
            return True
        return False
    literal, _ = random.choice(clauses)
    new_assignment = assignment.copy()
    new_assignment[literal] = True
    if dpll([c for c in clauses if literal not in c and -literal not in c], new_assignment):
        return True
    new_assignment[literal] = False
    if dpll([c for c in clauses if literal not in c and -literal not in c], new_assignment):
        return True
    return False

def resolution(clauses):
    while True:
        new_clauses = []
        for i in range(len(clauses)):
            for j in range(i+1, len(clauses)):
                clause_i = clauses[i]
                clause_j = clauses[j]
                common_literal = next((l for l in clause_i if -l in clause_j), None)
                if common_literal:
                    new_clause = [l for l in clause_i if l != common_literal] + [l for l in cl
                    if not new_clause:
                        return True
                    new_clauses.append(new_clause)
        if len(new_clauses) == len(clauses):
            return False
        clauses = new_clauses

def geometric_flow_invariant(n, variables, clauses):
    # Placeholder for actual implementation of geometric flow invariant
    return random.randint(1, n)

n = 20
variables, clauses = generate_tseitin_formula(n)
resolution_length = resolution(clauses)
rank_F_G = geometric_flow_invariant(n, variables, clauses)
ratio = resolution_length / (2 ** rank_F_G) if rank_F_G > 0 else float('inf')

return {
    "metric_name": "resolution_ratio",
    "metric_value": ratio,
```

```
        "instances_tested": 1,
        "conjecture_holds": ratio >= 1,  # Placeholder constant c
        "counterexample": "" if ratio >= 1 else "small_resolution_length"
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i + 1 for i in range(5, 8)]
    results = []
    total_ratio = 0
    count_supports_conjecture = 0

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        total_ratio += trial_result["metric_value"]
        if trial_result["conjecture_holds"]:
            count_supports_conjecture += 1
        results.append(trial_result)

    mean_ratio = total_ratio / len(results)
    support_fraction = count_supports_conjecture / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_ratio} std=0.0 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample='small_resolution_length' first_failing_seed={fir
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c134361c.py", line 103, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c134361c.py", line 83, in run_trial
    variables, clauses = generate_tseitin_formula(n)
                         ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c134361c.py", line 26, in generate_tsei
    clauses.append([-variables[i-1], f'y{i}'])
                    ^^^^^^^^^^^^^^^
TypeError: bad operand type for unary -: 'str'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Investigate and fix the error in the test code to ensure it can run to completion and produce results.

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	10564
2	propose	ollama_remote	glm4:latest	0	0	10579
3	propose	ollama_remote	glm4:latest	0	0	13307
4	propose	ollama_remote	glm4:latest	0	0	10397
5	preregistration	ollama_remote	glm4:latest	0	0	5705
6	novelty	ollama_remote	glm4:latest	0	0	4565
7	novelty	ollama_remote	glm4:latest	0	0	5735
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	44322
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13885
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12843
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12485
12	judge	ollama_remote	glm4:latest	0	0	12289

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 156675 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in `research/audit/22ad113ce145.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/22ad113ce145.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/22ad113ce145.tar.gz` (if generated)